1) #include <stdio.h>

```
int main(void)
{
    int price, qty, senior;
    int gross, wrong_total, wrong_net;
    double discount, tax, correct_total, correct_net;

    printf("Enter medicine price: ");
    scanf("%d", &price);

    printf("Enter quantity: ");
    scanf("%d", &qty);

    printf("Senior citizen? (1 = Yes, 0 = No): ");
    scanf("%d", &senior);

    gross = price * qty;

    /* Original incorrect calculation */
    wrong_total = price * qty
            - price * qty * 15 / 100 * senior
            + price * qty * 5 / 100;

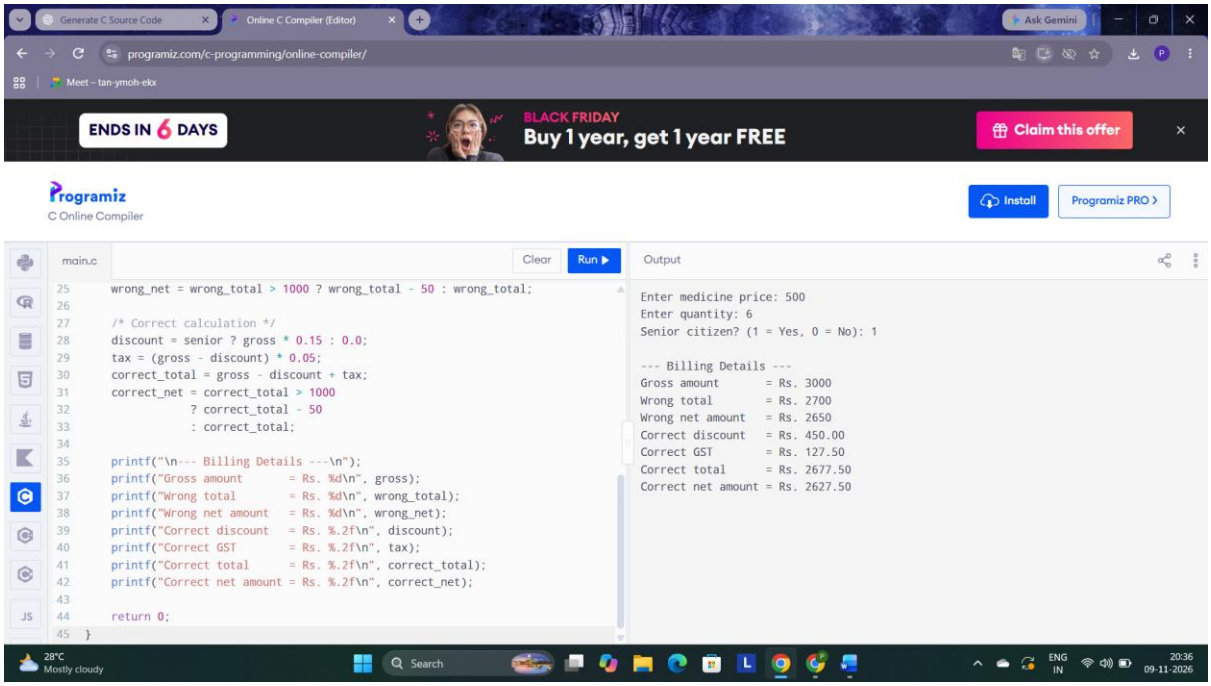
    wrong_net = wrong_total > 1000 ? wrong_total - 50 : wrong_total;

    /* Correct calculation */
    discount = senior ? gross * 0.15 : 0.0;
```

```
    tax = (gross - discount) * 0.05;
    correct_total = gross - discount + tax;
    correct_net = correct_total > 1000
            ? correct_total - 50
            : correct_total;

    printf("\n--- Billing Details ---\n");
    printf("Gross amount      = Rs. %d\n", gross);
    printf("Wrong total       = Rs. %d\n", wrong_total);
    printf("Wrong net amount   = Rs. %d\n", wrong_net);
    printf("Correct discount   = Rs. %.2f\n", discount);
    printf("Correct GST       = Rs. %.2f\n", tax);
    printf("Correct total     = Rs. %.2f\n", correct_total);
    printf("Correct net amount = Rs. %.2f\n", correct_net);

    return 0;
}
```



2) #include <stdio.h>

int calls = 0;

int verify_card(int card)

{

calls++;

return card;

}

int main(void)

{

int senior[1000];

int eligible[1000];

int card[1000];

int i, concession;

```
int order1, order2;

/* Create sample passenger data */
for (i = 0; i < 1000; i++) {
   senior[i] = (i % 2 == 0);
   eligible[i] = (i % 4 == 0);
   card[i] = (i % 10 == 0);
}

/* Order 1: Check senior and eligible first */
calls = 0;

for (i = 0; i < 1000; i++) {
   concession = (senior[i] && eligible[i])
         || verify_card(card[i]);
}

order1 = calls;

/* Order 2: Check card first */
calls = 0;

for (i = 0; i < 1000; i++) {
   concession = verify_card(card[i])
         || (senior[i] && eligible[i]);
}

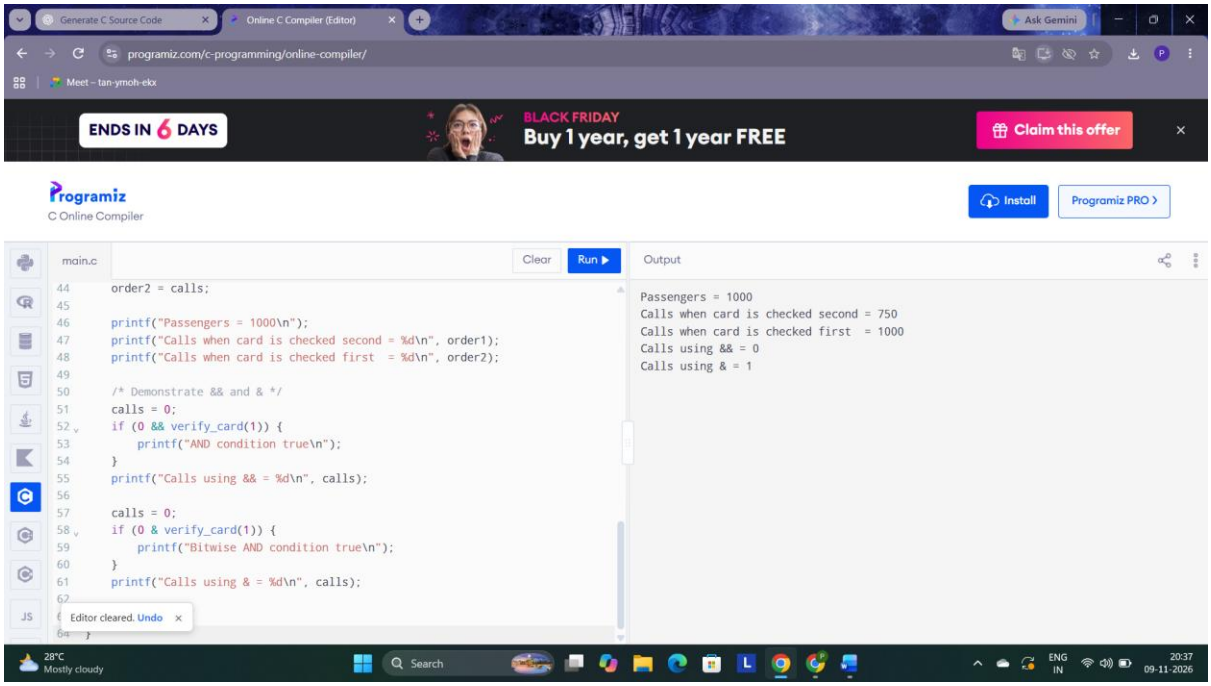
order2 = calls;
```

```
    printf("Passengers = 1000\n");
    printf("Calls when card is checked second = %d\n", order1);
    printf("Calls when card is checked first  = %d\n", order2);

    /* Demonstrate && and & */
    calls = 0;
    if (0 && verify_card(1)) {
        printf("AND condition true\n");
    }
    printf("Calls using && = %d\n", calls);

    calls = 0;
    if (0 & verify_card(1)) {
        printf("Bitwise AND condition true\n");
    }
    printf("Calls using & = %d\n", calls);

    return 0;
}
```



3) #include <stdio.h>

```
enum {
   LIGHT = 0,
   FAN = 1,
   AC = 2,
   HEATER = 3,
   PUMP = 4
};

unsigned char switch_on(unsigned char reg, int dev)
{
   return reg | (1u << dev);
}

unsigned char switch_off(unsigned char reg, int dev)
```

```
{
   return reg & ~(1u << dev);
}

unsigned char toggle(unsigned char reg, int dev)
{
   return reg ^ (1u << dev);
}

int is_on(unsigned char reg, int dev)
{
   return (reg & (1u << dev)) != 0;
}

void show(unsigned char reg)
{
   int bit;

   for (bit = 7; bit >= 0; bit--) {
      putchar(((reg >> bit) & 1u) ? '1' : '0');
   }

   putchar('\n');
}

int main(void)
{
   unsigned char reg = 0;
```

```
    reg = switch_on(reg, LIGHT);
    reg = switch_on(reg, AC);

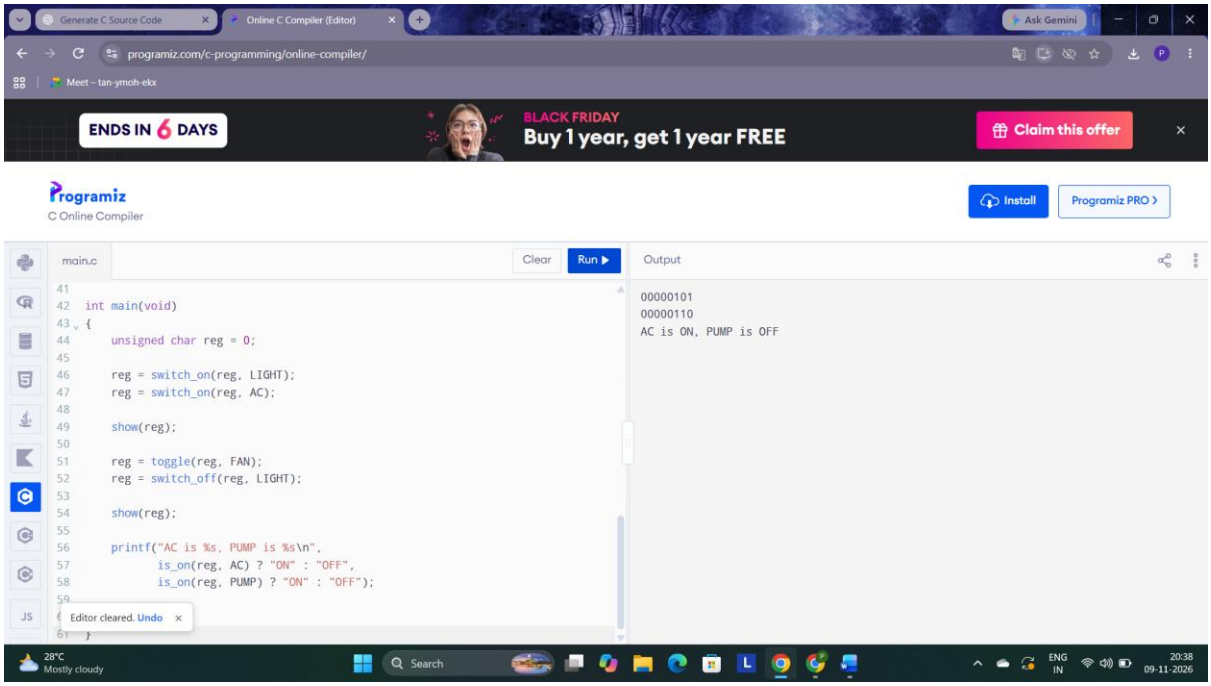
    show(reg);

    reg = toggle(reg, FAN);
    reg = switch_off(reg, LIGHT);

    show(reg);

    printf("AC is %s, PUMP is %s\n",
        is_on(reg, AC) ? "ON" : "OFF",
        is_on(reg, PUMP) ? "ON" : "OFF");

    return 0;
}
```



4) #include <stdio.h>

```
char grade(int marks)
{
   if (marks < 0 || marks > 100) {
      return '?';
   }

   switch (marks / 10) {
      case 10:
      case 9:
         return 'S';

      case 8:
         return 'A';

      case 7:
         return 'B';

      case 6:
         return 'C';

      case 5:
         return 'D';

      default:
         return 'F';
   }
```

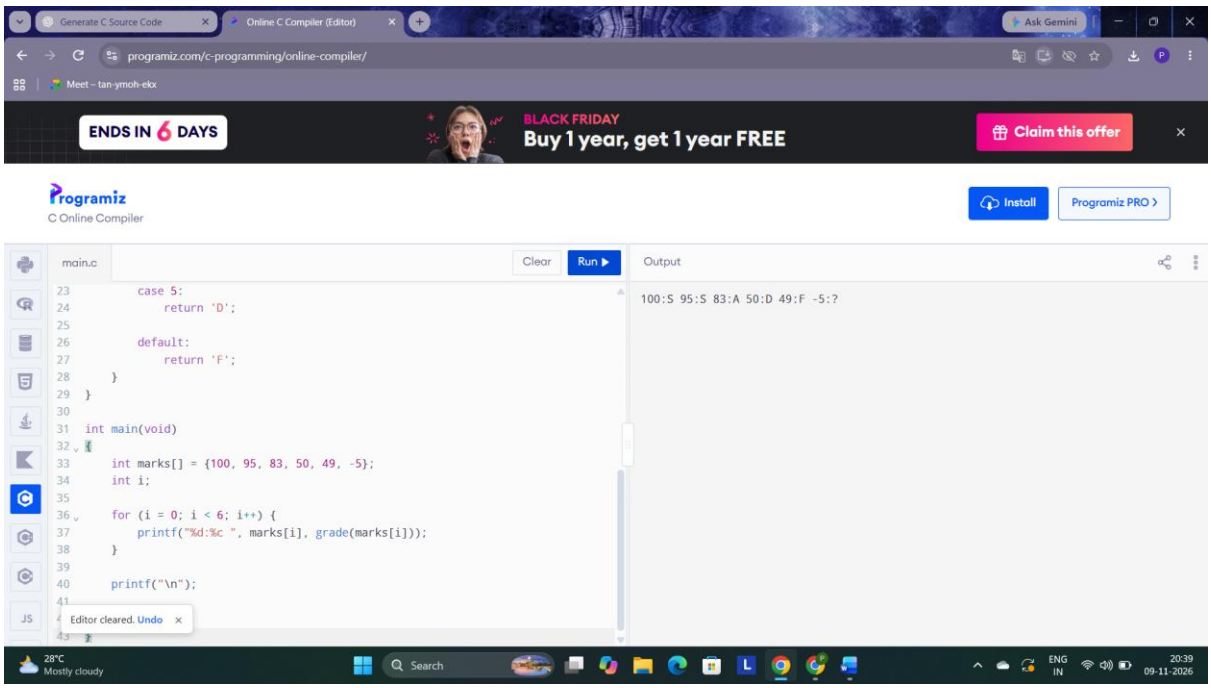
```
}

int main(void)
{
   int marks[] = {100, 95, 83, 50, 49, -5};
   int i;

   for (i = 0; i < 6; i++) {
      printf("%d:%c ", marks[i], grade(marks[i]));
   }

   printf("\n");

   return 0;
}
```



5) #include <stdio.h>

```
unsigned long checks;

/* Original method: test divisors up to n - 1 */
int prime_original(unsigned long n)
{
   unsigned long i;

   checks = 0;

   if (n < 2) {
      return 0;
   }

   for (i = 2; i < n; i++) {
      checks++;

      if (n % i == 0) {
         return 0;
      }
   }

   return 1;
}

/* Improved method using for loop */
int prime_for(unsigned long n)
{
   unsigned long i;
```

```
    checks = 0;

    if (n < 2) {
        return 0;
    }

    for (i = 2; i <= n / i; i++) {
        checks++;

        if (n % i == 0) {
            return 0;
        }
    }

    return 1;
}

/* Improved method using while loop */
int prime_while(unsigned long n)
{
    unsigned long i = 2;

    checks = 0;

    if (n < 2) {
        return 0;
    }
```

```
    while (i <= n / i) {
        checks++;

        if (n % i == 0) {
            return 0;
        }

        i++;
    }

    return 1;
}

/* Improved method using do-while loop */
int prime_do_while(unsigned long n)
{
    unsigned long i = 2;

    checks = 0;

    if (n < 2) {
        return 0;
    }

    do {
        if (i > n / i) {
            break;
```

```
        }

        checks++;

        if (n % i == 0) {
            return 0;
        }

        i++;

    } while (1);

    return 1;
}

int main(void)
{
    unsigned long tickets[] = {97, 100, 7919, 1000003};
    int i, result;

    printf("--- Improved for loop ---\n");

    for (i = 0; i < 4; i++) {
        result = prime_for(tickets[i]);

        printf("%lu: %s, divisions = %lu\n",
               tickets[i],
               result ? "prime" : "not prime",
```

```
        checks);
}

printf("\n--- Improved while loop ---\n");

for (i = 0; i < 4; i++) {
   result = prime_while(tickets[i]);

   printf("%lu: %s, divisions = %lu\n",
        tickets[i],
        result ? "prime" : "not prime",
        checks);
}

printf("\n--- Improved do-while loop ---\n");

for (i = 0; i < 4; i++) {
   result = prime_do_while(tickets[i]);

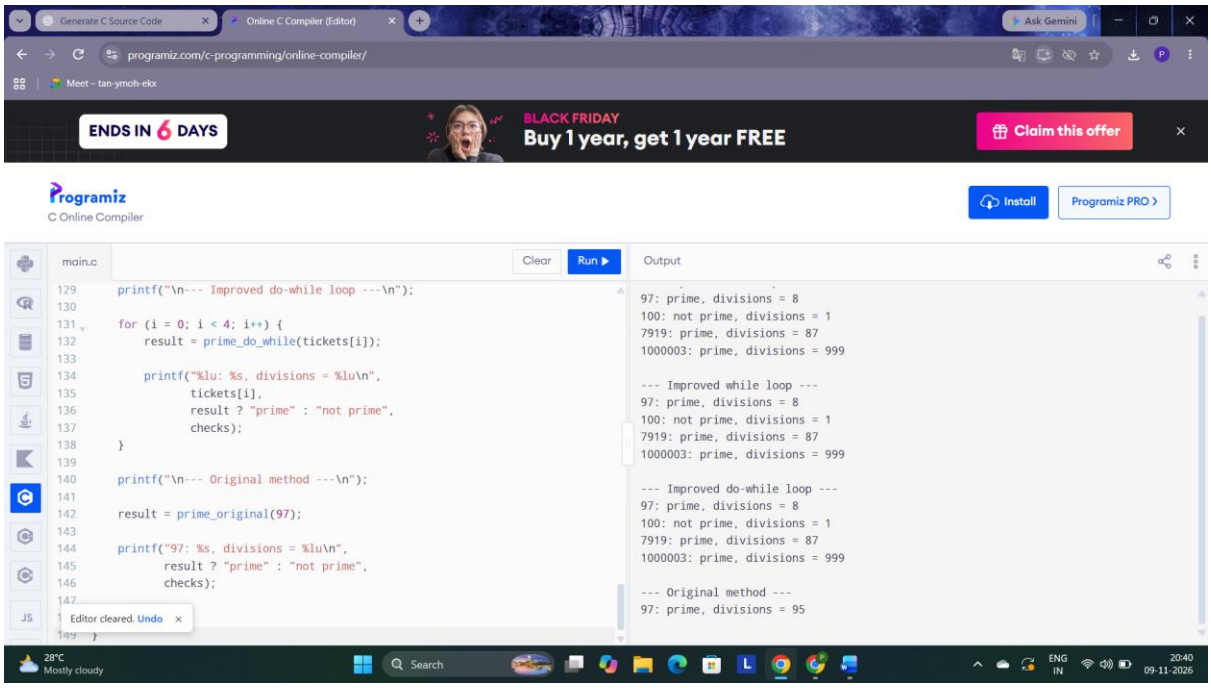
   printf("%lu: %s, divisions = %lu\n",
        tickets[i],
        result ? "prime" : "not prime",
        checks);
}

printf("\n--- Original method ---\n");

result = prime_original(97);
```

```
    printf("97: %s, divisions = %lu\n",
           result ? "prime" : "not prime",
           checks);

    return 0;
}
```



6) #include <stdio.h>

```
int main(void)
{
    double balance = 10000.0;
    double repayment = 2000.0;
    double rate = 0.02;
```

```
double interest;
double amount_due;
double total_interest = 0.0;
double payment;

int month = 0;

printf("Month\tOpening Balance\tInterest\tPayment\tClosing Balance\n");

while (balance > 0.005 && month < 360) {
   month++;

   interest = balance * rate;
   total_interest += interest;

   amount_due = balance + interest;

   if (repayment <= interest) {
      printf("\nLoan cannot be fully repaid with this repayment amount.\n");
      return 0;
   }

   if (repayment < amount_due) {
      payment = repayment;
   } else {
      payment = amount_due;
   }
```

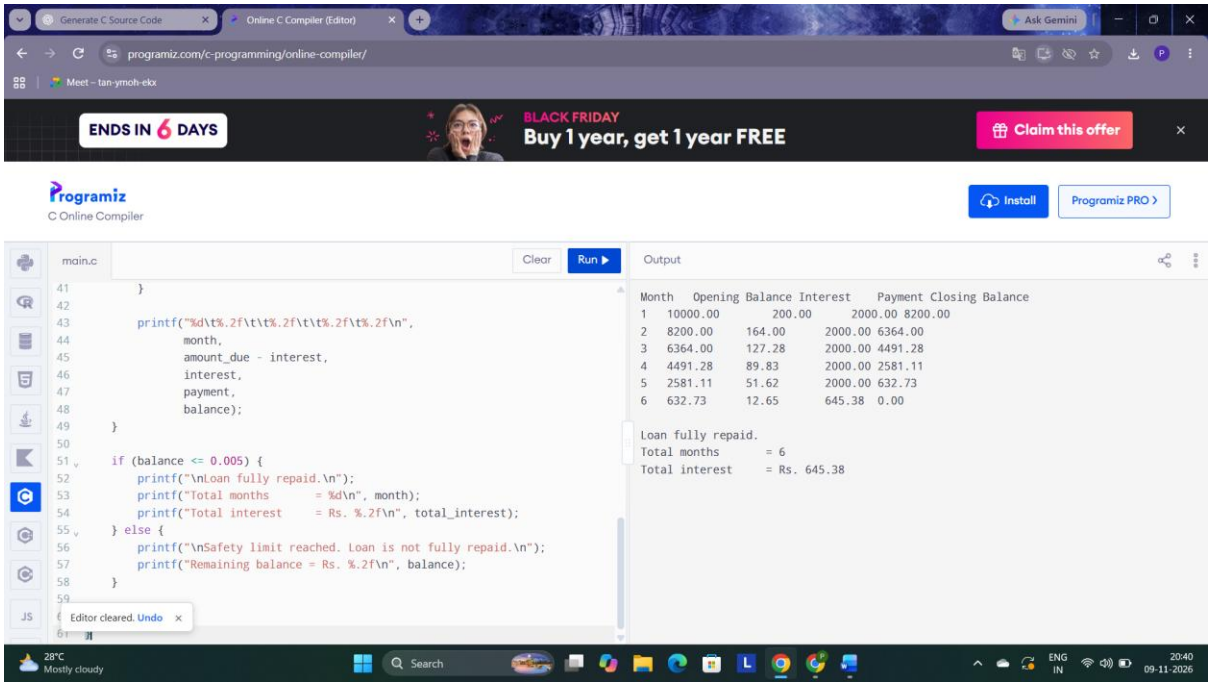
```
        balance = amount_due - payment;

        if (balance < 0.005) {
            balance = 0.0;
        }

        printf("%d\t%.2f\t\t%.2f\t\t%.2f\t%.2f\n",
               month,
               amount_due - interest,
               interest,
               payment,
               balance);
    }

    if (balance <= 0.005) {
        printf("\nLoan fully repaid.\n");
        printf("Total months      = %d\n", month);
        printf("Total interest    = Rs. %.2f\n", total_interest);
    } else {
        printf("\nSafety limit reached. Loan is not fully repaid.\n");
        printf("Remaining balance = Rs. %.2f\n", balance);
    }

    return 0;
}
```



7) #include <stdio.h>

unsigned long checks = 0;

int is_prime(unsigned long n)

{

unsigned long i;

checks = 0;

if (n < 2) {

return 0;

}

if (n == 2) {

return 1;

}

```
    if (n % 2 == 0) {
        checks++;
        return 0;
    }

    for (i = 3; i <= n / i; i += 2) {
        checks++;

        if (n % i == 0) {
            return 0;
        }
    }

    return 1;
}

int main(void)
{
    unsigned long numbers[] = {97, 100, 7919, 1000003};
    int i, result;

    printf("Improved Prime Number Program\n\n");

    for (i = 0; i < 4; i++) {
        result = is_prime(numbers[i]);

        printf("%lu is %s\n",
```

```
            numbers[i],
            result ? "Prime" : "Not Prime");

        printf("Number of divisions = %lu\n\n", checks);
    }

    return 0;
}
```

